DevOps Shack

DOCKER CHEAT SHEET

100+ Essential Commands & Concepts

BY DEVOPS SHACK

[Click here for DevSecOps & Cloud DevOps Course]

DevOps Shack

Docker Cheat Sheet:

100+ Essential Commands & Concepts

Introduction

Docker is the cornerstone of modern DevOps, enabling developers to package applications with their dependencies into lightweight containers. These containers run consistently across environments, making Docker essential for CI/CD pipelines, microservices, and cloud deployments.

This cheat sheet provides over **100 Docker commands** — from installation and basic usage to advanced networking, volumes, and orchestration. Whether you're new to Docker or managing production workloads, this guide is designed as a quick reference for daily use.

1. Docker Basics

These commands are used when first setting up Docker or inspecting the environment. You'll verify versions, log in to Docker Hub, search and pull base images, and inspect them. They're essential for preparing your environment before working with containers.

Command	Description
docker --version	Show Docker version
docker info	Show system-wide Docker info
docker login	Log in to Docker Hub
docker logout	Log out from Docker Hub
docker search <image>	Search for images on Docker Hub
docker pull <image>	Download an image
docker images	List local images
docker rmi <image>	Remove image

Command	Description
docker inspect <image>	Show detailed info about an image
docker history <image>	Show image history and layers

2. Container Management

These commands handle the full lifecycle of containers — starting, stopping, restarting, and removing them. You'll use them daily to manage services in dev/test environments and keep your workspace clean.

Command	Description
docker run <image>	Run a container from image
docker run -d <image>	Run container in detached mode
docker run -it <image> bash	Run container interactively with shell
docker ps	List running containers
docker ps -a	List all containers
docker stop <id>	Stop a running container
docker start <id>	Start a stopped container
docker restart <id>	Restart container
docker rm <id>	Remove container
docker container prune	Remove all stopped containers

3. Working Inside Containers

These commands are used to debug or interact with running containers. They let you run commands inside, view logs, copy files, and inspect processes. They're essential for troubleshooting application issues in real time.

Command	Description
docker exec -it <id> bash	Open interactive shell in container

Command	Description
docker exec <id> ls /	Run command inside container
docker attach <id>	Attach to running container
docker cp <id>:/file /host/path	Copy file from container to host
docker cp /host/file <id>:/path	Copy file from host to container
docker logs <id>	Show container logs
docker logs -f <id>	Follow container logs in real time
docker top <id>	Show processes running in container
docker stats	Show live CPU/memory usage of containers
docker inspect <id>	Show detailed container info

4. Docker Images

Treat images as immutable build artifacts. Build, tag, push/pull, and clean up. Use these to version your app and move artifacts through environments (dev → stage → prod).

Command	Description
docker build -t myapp:latest .	Build from Dockerfile
docker build --no-cache -t myapp:clean .	Build without cache
docker tag myapp:latest user/repo:1.0.0	Tag image
docker push user/repo:1.0.0	Push to registry
docker pull user/repo:1.0.0	Pull from registry
docker image rm <image>	Remove image
docker image prune	Remove unused images
docker save -o myapp.tar myapp:latest	Save image to tar
docker load -i myapp.tar	Load image from tar

Command	Description
docker image inspect <image>	Inspect image metadata

5. Volumes & Bind Mounts

Persist data across restarts and share files between host and container. Use **named volumes** for app data (DBs, caches) and **bind mounts** for local dev (hot-reload).

Command	Description
docker volume create myvol	Create volume
docker volume ls	List volumes
docker volume inspect myvol	Inspect volume
docker volume rm myvol	Remove volume
docker volume prune	Remove unused volumes
docker run -v myvol:/data <img>	Attach named volume
docker run -v /host:/cont <img>	Bind mount host dir
docker run --mount type=volume,src=myvol,dst=/data <img>	Modern volume mount
docker run --mount type=bind,src=$PWD,dst=/app <img>	Modern bind mount
docker cp <id>:/data ./backup/	Quick data copy/backup

6. Networking

Connect services and expose ports. Use user-defined bridge networks for microservices, port mappings for local access, and isolate workloads for security.

Command	Description
docker network ls	List networks
docker network create mynet	Create network
docker network inspect mynet	Inspect network
docker network rm mynet	Remove network
docker network prune	Remove unused networks
docker run --network=mynet <img>	Attach to network
docker network connect mynet <id>	Connect container
docker network disconnect mynet <id>	Disconnect container
docker run -p 8080:80 <img>	Map host:container ports
docker port <id>	Show port mappings

7. Docker Compose

Define multi-container apps declaratively (docker-compose.yml). Great for local dev stacks and reproducible environments in CI.

Command	Description
docker compose version	Show Compose version
docker compose up -d	Start all services detached
docker compose down	Stop and remove services
docker compose down -v	Also remove named volumes
docker compose ps	List services/containers
docker compose logs -f	Follow service logs
docker compose exec <svc> sh	Exec into a service
docker compose build	Build images for services
docker compose pull	Pull service images

Command	Description
docker compose config	Validate & view merged config

8. Docker Swarm (Orchestration)

Built-in orchestrator for clustering, services, and rolling updates. Handy for simple HA setups and demos when K8s is overkill.

Command	Description
docker swarm init	Initialize Swarm (manager)
docker swarm join-token worker	Get worker join token
docker swarm leave --force	Leave swarm
docker node ls	List swarm nodes
docker service create --name web -p 80:80 nginx	Create service
docker service ls	List services
docker service ps web	Tasks for a service
docker service scale web=5	Scale replicas
docker stack deploy -c stack.yml mystack	Deploy a stack
docker stack rm mystack	Remove a stack

9. Security & Best Practices

Run containers with least privilege and add health checks. Use these flags in CI/CD and prod to reduce blast radius and improve reliability.

Command	Description
docker scan <image>	Scan image for vulns (Snyk integration)
docker run --read-only <img>	Filesystem read-only
docker run --user 1001:1001 <img>	Drop root; run as UID:GID

Command	Description
docker run --cap-drop ALL <img>	Drop all Linux capabilities
docker run --security-opt no-new-privileges <img>	Block priv escalation
docker run --pids-limit 200 <img>	Limit process count
docker run --memory 512m --cpus 1 <img>	Limit resources
docker run --network none <img>	Disable networking
`docker run --health-cmd="curl -f http://localhost/	
docker image ls --digests	View image digests for pinning

10. Debugging, System & Maintenance

Keep Docker healthy, free disk space, and diagnose issues. These are your go-to commands when “it worked yesterday”.

Command	Description
docker events	Live Docker daemon events
docker system df	Disk usage by images/containers
docker system prune -f	Clean unused objects
docker builder prune -f	Clean build cache
docker inspect <id>	Deep JSON details
docker diff <id>	FS changes in container
docker kill <id>	Send SIGKILL to container
docker pause <id> / docker unpause <id>	Pause/unpause processes
journalctl -u docker -f	Follow Docker daemon logs (Linux)
dockerd --debug	Start daemon in debug (manual envs)

Docker Quick Reference

Category	Command	Description
Basics	docker --version	Show Docker version
	docker info	Show system info
	docker login	Log in to Docker Hub
	docker pull nginx	Pull an image
	docker images	List images
Containers	docker run nginx	Run container
	docker run -d -p 8080:80 nginx	Run detached + map port
	docker ps	List running containers
	docker ps -a	List all containers
	docker stop <id>	Stop container

Category	Command	Description
	docker start <id>	Start container
	docker rm <id>	Remove container
Inside Containers	docker exec -it <id> bash	Open shell
	docker logs <id>	Show logs
	docker logs -f <id>	Follow logs
	docker cp <id>:/src ./dest	Copy file from container
	docker stats	Show live resource usage
Images	docker build -t myapp .	Build image from Dockerfile
	docker tag myapp user/repo:1.0	Tag image
	docker push user/repo:1.0	Push image
	docker rmi <image>	Remove image
	docker image prune	Clean unused images
Volumes	docker volume create data	Create volume
	docker run -v data:/app nginx	Mount volume
	docker volume ls	List volumes
	docker volume prune	Remove unused volumes
Networking	docker network ls	List networks
	docker network create mynet	Create network
	docker run --network=mynet nginx	Connect container to network
	docker port <id>	Show port mappings
Compose	docker compose up -d	Start services
	docker compose down	Stop + remove services

Category	Command	Description
	docker compose logs -f	Follow service logs

Conclusion

Docker has transformed the way we build, ship, and run applications. By mastering its commands — from pulling images and running containers to managing networks, volumes, and orchestration — you gain the ability to create lightweight, portable, and consistent environments across development, testing, and production.

This cheat sheet condenses more than 100 essential Docker commands into practical categories, making it easier to find exactly what you need when you need it. The one-page quick reference serves as a fast daily lookup, while the full guide provides context and best practices for real-world use.

Whether you are debugging a failing container, optimizing images for CI/CD pipelines, or deploying multi-service applications, Docker equips you with the flexibility and speed to move from code to production seamlessly. With these commands at your fingertips, you're ready to handle everything from simple single-container apps to complex distributed systems.

Docker isn't just a container engine — it's a cornerstone of modern DevOps, and mastering it puts you a step closer to mastering cloud-native development.